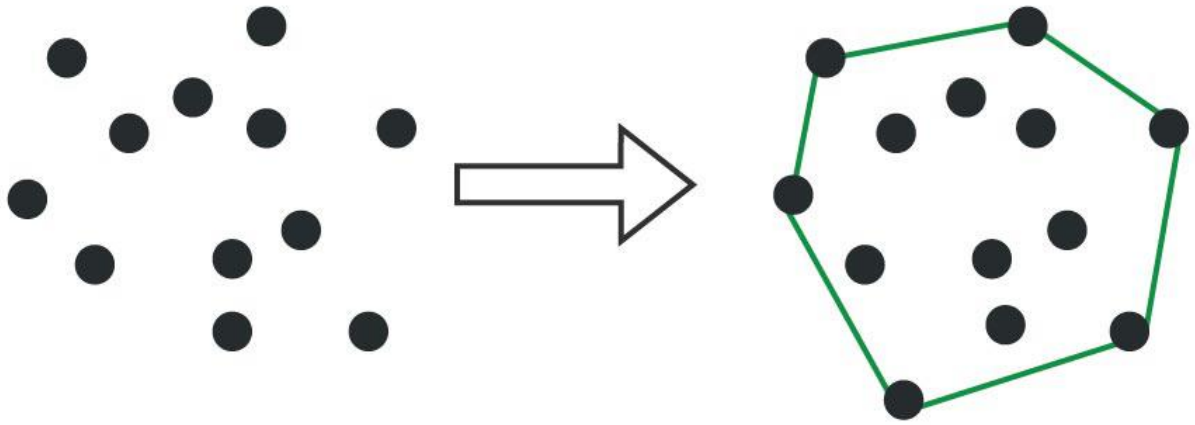
Convex Hull using Graham Scan

Last Updated : 23 Jul, 2025

A **convex hull** is the smallest convex polygon that contains a given set of points. It is a useful concept in computational geometry and has applications in various fields such as **computer graphics, image processing, and collision detection**.

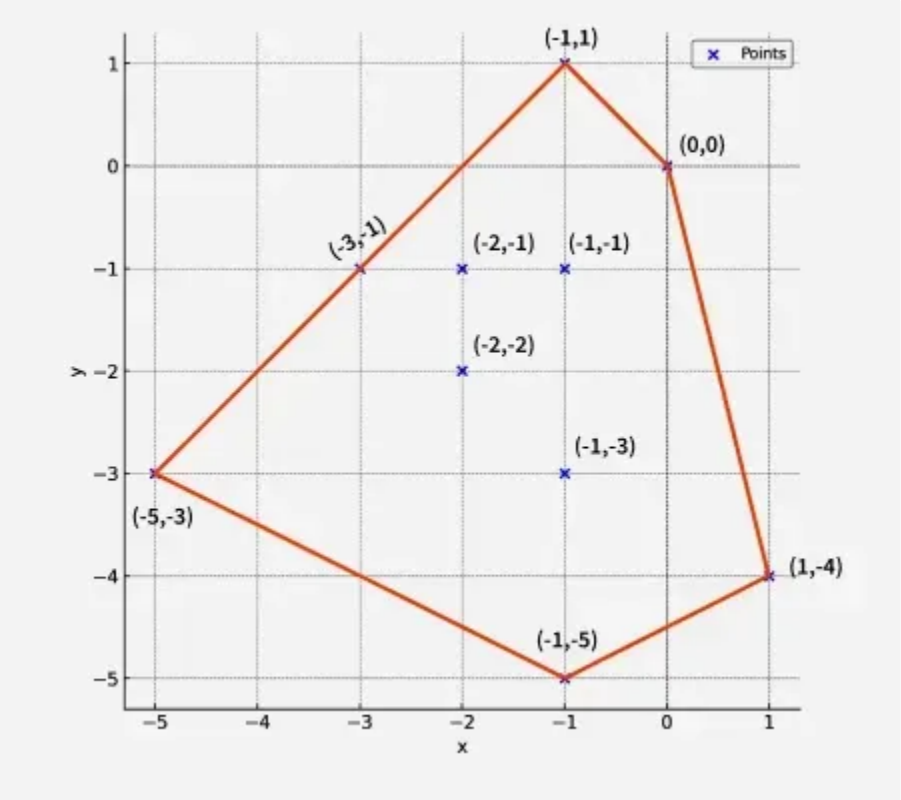
A convex polygon is a polygon in which all interior angles are less than **180degrees**. A convex hull can be constructed for any set of points, regardless of their arrangement.



Convex Hull

Examples

Input: *points[][] = [[0, 0], [1, -4], [-1, -5], [-5, -3], [-3, -1], [-1, -3], [-2, -2], [-1, -1], [-2, -1], [-1, 1]]*
Output: *[[-5, -3], [-1, 1], [0, 0], [1, -4], [-1, -5]*
Explantation: *The figure below shows the points of a convex polygon. These points define the boundary of the polygon.*



Approach:

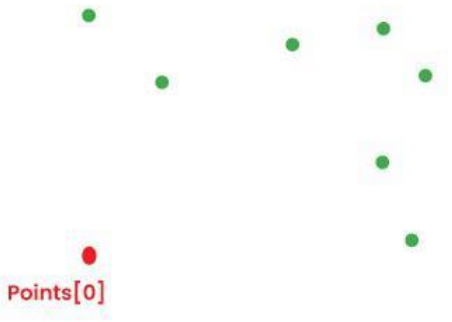
Prerequisite: How to check if two given line segments intersect?

*The **Graham scan algorithm** is a simple and efficient algorithm for computing the convex hull of a set of points. It works by iteratively adding points to the convex hull until all points have been added.*

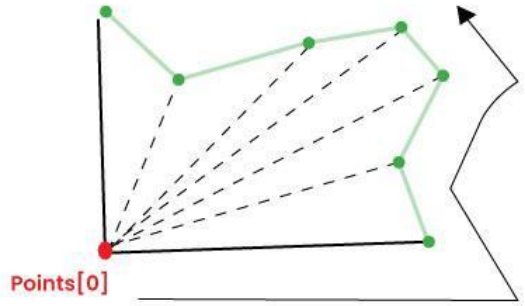
- *The algorithm starts by finding the point with the smallest y-coordinate. This point is always on the convex hull. The algorithm then sorts the remaining points by their polar angle with respect to the starting point.*
- *The algorithm then iteratively adds points to the **convex hull**. At each step, the algorithm checks whether the last two points added to the convex hull form a right turn. If they do, then the last point is removed from the convex hull. Otherwise, the next point in the sorted list is added to the convex hull.*

Step by Step Approach

Phase 1 (Sort points): The first step of the Graham Scan algorithm is to sort the points by their polar angle relative to the starting point. After sorting, the starting point is added to the convex hull, and the sorted points form a simple closed path.



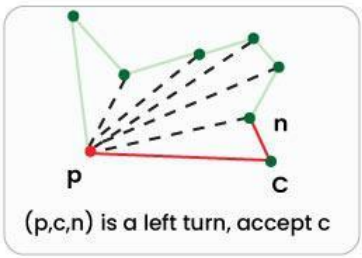
Given points



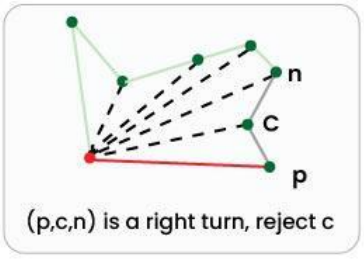
Points consider according to increasing angle with respect to point [0] from a simple closed path

Convex Hull using Graham Scan

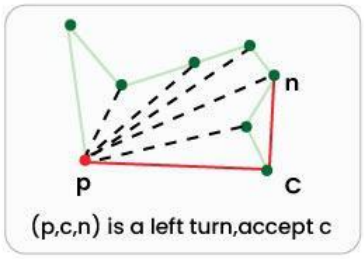
Phase 2 (Accept or Reject Points): After forming the closed path, we traverse it to remove concave points. Using orientation, we keep the first two points and check the next point by considering the last three points be **prev(p)**, **curr(c)** and **next(n)**. If the angle formed by these three points is not counterclockwise, we discard **(reject)** the current point, otherwise, we keep **(accept)** it.



(p,c,n) is a left turn, accept c

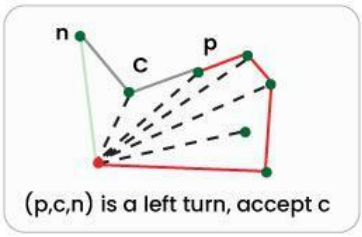


(p,c,n) is a right turn, reject c

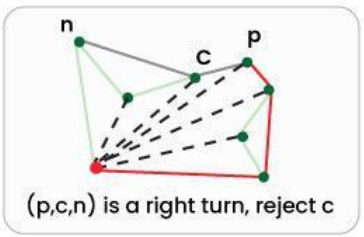


(p,c,n) is a left turn,accept c

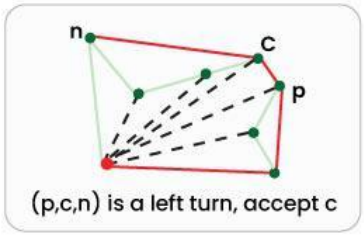
P: previouse
c: current
n : next



(p,c,n) is a left turn, accept c



(p,c,n) is a right turn, reject c



(p,c,n) is a left turn, accept c

In the above algorithm and below code, a stack of points is used to store convex hull points. With reference to the code,p is next-to-top in stack, c is top of stack and n is point[i]

Convex Hull using Graham Scan

C++ Java Python C# JavaScript

```cpp
#include <bits/stdc++.h>
using namespace std;

 // Structure to represent a point
struct Point {
    double x, y;

    // Operator to check equality of two points
    bool operator==(const Point& t) const {
        return x == t.x && y == t.y;
    }
};

 // Function to find orientation of the triplet (a, b, c)
 // Returns -1 if clockwise, 1 if counter-clockwise, 0 if
collinear
int orientation(Point a, Point b, Point c) {
    double v = a.x * (b.y - c.y) +
               b.x * (c.y - a.y) +
               c.x * (a.y - b.y);
    if (v < 0) return -1;
    if (v > 0) return +1;
    return 0;
}

 // Function to calculate the squared distance between two points
double distSq(Point a, Point b) {
    return (a.x - b.x) * (a.x - b.x) +
           (a.y - b.y) * (a.y - b.y);
}

 // Function to find the convex hull of a set of 2D points
vector<vector<int>> findConvexHull(vector<vector<int>> points) {

    // Store number of points points
    int n = points.size();

    // Convex hull is not possible if there are fewer than 3
points
    if (n < 3) return {{-1}};

    // Convert points 2D vector into vector of Point structures
```

```
    vector<Point> a;
    for (auto& p : points) {
        a.push_back({(double)p[0], (double)p[1]});
    }

    // Find the point with the lowest y-coordinate (and leftmost
in case of tie)
    Point p0 = *min_element(a.begin(), a.end(), [](Point a, Point
b) {
        return make_pair(a.y, a.x) < make_pair(b.y, b.x);
    });

    // Sort points based on polar angle with respect to the
reference point p0
    sort(a.begin(), a.end(), [&p0](const Point& a, const Point& b)
{
        int o = orientation(p0, a, b);

        // If points are collinear, keep the farthest one last
        if (o == 0) {
            return distSq(p0, a) < distSq(p0, b);
        }

        // Otherwise, sort by counter-clockwise order
        return o < 0;
    });

    // Vector to store the points on the convex hull
    vector<Point> st;

    // Process each point to build the hull
    for (int i = 0; i < (int)a.size(); ++i) {

        // While last two points and current point make a non-left
turn, remove the middle one
        while (st.size() > 1 && orientation(st[st.size() - 2],
st.back(), a[i]) >= 0)
            st.pop_back();

        // Add the current point to the hull
        st.push_back(a[i]);
    }
```

```
    // If fewer than 3 points in the final hull, return {-1}
    if (st.size() < 3) return {{-1}};

    // Convert the final hull into a vector of vectors of integers
    vector<vector<int>> result;
    for (auto& p : st) {
        result.push_back({(int)p.x, (int)p.y});
    }

    return result;
}

int main() {

    // Define the points set of 2D points
    vector<vector<int>> points = {
        {0, 0}, {1, -4}, {-1, -5}, {-5, -3}, {-3, -1},
        {-1, -3}, {-2, -2}, {-1, -1}, {-2, -1}, {-1, 1}
    };

    // Call the function to compute the convex hull
    vector<vector<int>> hull = findConvexHull(points);

    // If hull contains only {-1}, print the error result
    if(hull.size() == 1 && hull[0].size() == 1){
        cout << hull[0][0] << " ";
    }
    else {

        // Print each point on the convex hull
        for (auto& point : hull) {
            cout << point[0] << ", " << point[1] << "\n";
        }
    }

    return 0;
}
```

Output

```
-1 -5
1 -4
0 0
-3 -1
-5 -3
```

Time Complexity: O(n log n), for finding the bottom-most point takes O(n), sorting the points takes O(n log n), and building the hull through stack operations takes O(n).
Space Complexity: O(n), due to the stack used for storing the points during the hull construction, with no significant additional space required.

Applications of Convex Hulls:

Convex hulls have a wide range of applications, including:

- **Collision detection:** Convex hulls can be used to quickly determine whether two objects are colliding. This is useful in computer graphics and physics simulations.
- **Image processing:** Convex hulls can be used to segment objects in images. This is useful for tasks such as object recognition and tracking.
- **Computational geometry:** Convex hulls are used in a variety of computational geometry algorithms, such as finding the closest pair of points and computing the diameter of a set of points.

Graham Scan Algorithm

Article Tags : Mathematical Geometric DSA Morgan Stanley +1 More

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem